
AI infrastructure follows the workload

From graphics processors to matrix engines, distributed training, automated co-design, and agentic systems

Li Shang

ML Systems • September 2026

Li Shang

THE MAIN POINT

Workloads redefine the scarce resource

- Graphics rewarded parallel processing.
- Deep learning rewarded matrix throughput and data reuse.
- Large-model training made memory, communication, and reliability system problems.
- Inference elevated bandwidth and latency.
- Agentic AI now combines accelerator work with CPU control, retrieval, communication, and external tools.

Graphics created a massively parallel processor

The first GPU workload was broader than matrix multiplication.

Graphics pipeline

Geometry transformed vertices. Rasterization, texture access, depth testing, and pixel shading produced the final image.

Data parallelism

The processor applied similar operations to many vertices or pixels at once. This created far more arithmetic throughput than a CPU.

Growing programmability

Larger transistor budgets allowed fixed graphics stages to become programmable, preparing the GPU for workloads beyond graphics.

CUDA made parallel hardware usable beyond graphics

Hardware throughput became broadly useful only after the software stack changed.

Before CUDA

Researchers used programmable shaders for science and finance, but often had to express non-graphics problems through graphics interfaces.

CUDA programming model

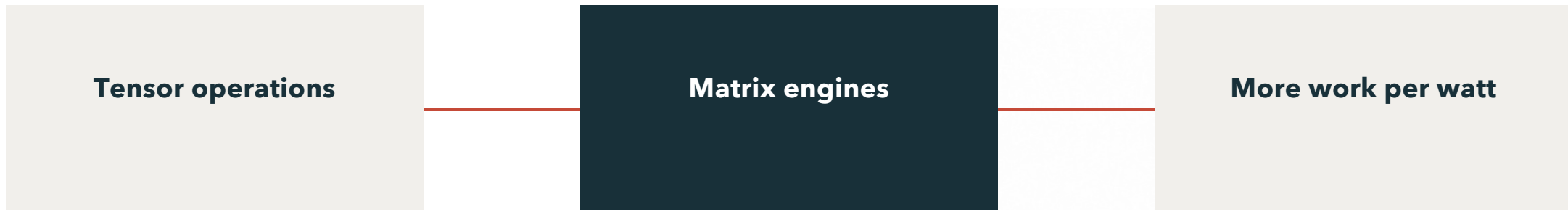
CUDA exposed threads, memory spaces, synchronization, a compiler, optimized libraries, and debugging tools for general parallel programs.

Hardware and software together

NVIDIA also changed the GPU to support general computation. The combined platform, rather than one instruction set, enabled widespread adoption.

Deep learning changed the dominant computation

Dense neural networks repeatedly reuse values inside matrix products.



Linear layers, convolutions, and attention create large collections of multiply-and-accumulate operations.

Tensor Cores, TPU matrix units, and Ascend Cube units keep reused values close to the arithmetic units.

The result is much higher dense tensor throughput than a general-purpose processor can provide at similar power.

The three designs share a workload principle, but their array sizes, dataflows, formats, and software models differ.

Lower precision changes compute and data movement

A numerical format helps only when the model keeps acceptable accuracy.

Wider formats

What they provide

FP32 offers range and precision that simplify training and sensitive numerical operations.

Cost

Each value consumes more storage, bandwidth, multiplier area, and energy.

Lower precision

What it provides

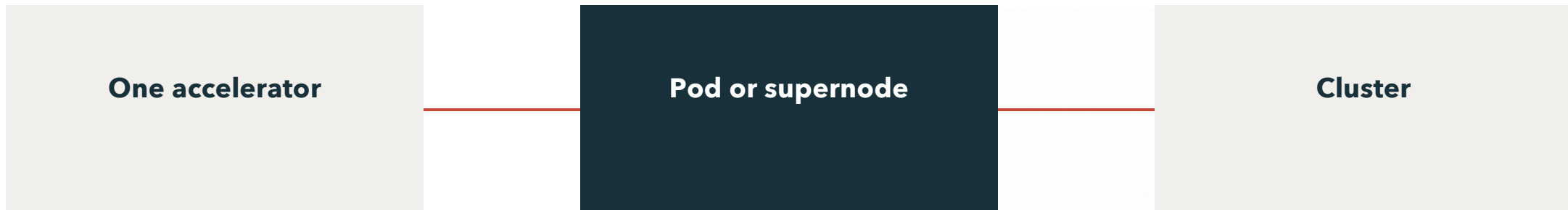
BF16, FP8, INT8, or FP4 can move and process more values per cycle.

Cost

Training and inference may need wider accumulation, scaling, calibration, or quantization-aware methods.

Large-model training turned the accelerator into a distributed system

The model, optimizer state, and activations may no longer fit on one device.



Local matrix engines process the model shard and retain intermediate values for gradient computation.

High-bandwidth links combine gradients and move activations among tightly connected accelerators.

Scale-out networking, storage, checkpointing, scheduling, and fault recovery keep long jobs progressing.

Adding accelerators helps only when useful computation grows faster than communication and recovery overhead.

Google TPU and Huawei Ascend followed the same workload logic

Both platforms balance matrix computation with memory, control, and software.

Google TPU

Core design

A large systolic matrix unit and compiler-managed dataflow target regular tensor operations.

System evolution

Later generations added SparseCores, high-bandwidth memory, custom interconnects, and optical switching.

Huawei Ascend

Core design

Da Vinci combines Cube units for matrices, Vector units for other tensor work, and Scalar units for control.

System evolution

Later generations expand memory, interconnect, precision formats, and supernode-scale integration.

Training and inference now reward different designs

The same model creates different system pressures during learning and serving.

Training

Workload

Large batches provide parallel work. The system retains activations and exchanges gradients across many devices.

Design pressure

Memory capacity, collective communication, reliable scale, and sustained throughput matter most.

Inference

Workload

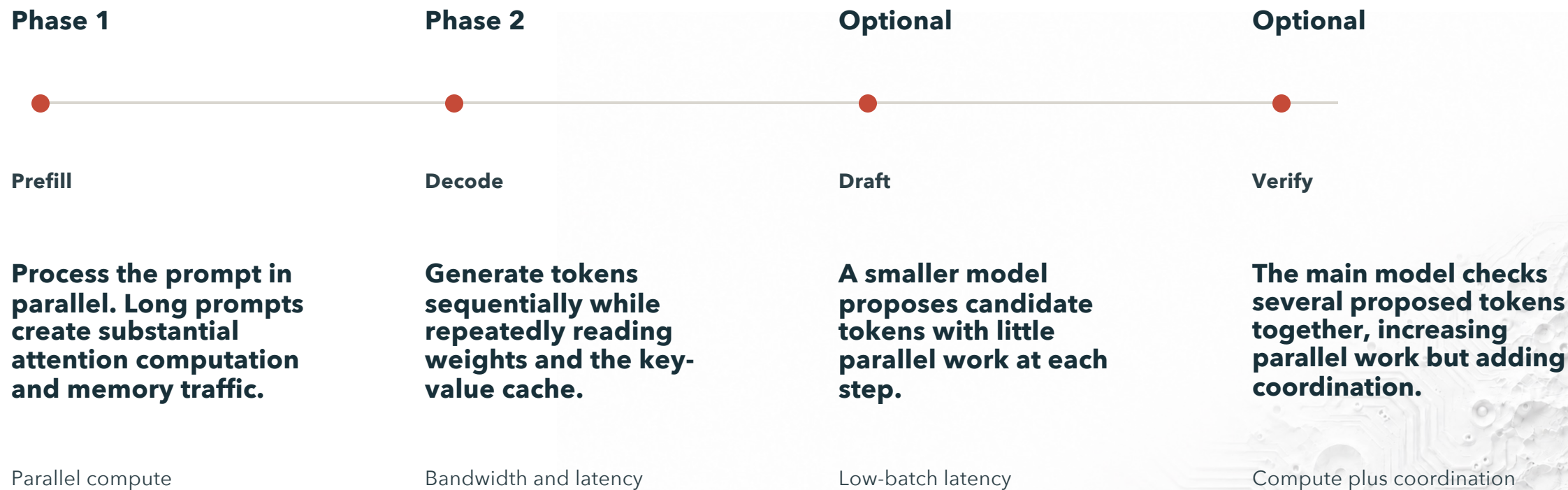
Token generation repeatedly reads weights and the key-value cache, often with small or variable batches.

Design pressure

Memory bandwidth, cache capacity, synchronization, routing, and predictable latency become central.

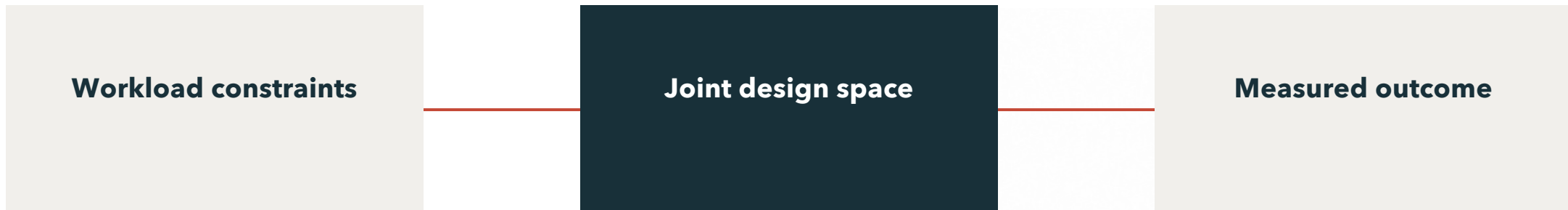
Modern inference contains several different workloads

One request can move through phases with different bottlenecks.



Hardware and software decisions must be optimized together

A choice in one layer changes the best choice in another.



Model shape, batch size, latency target, accuracy, and communication pattern define the real objective.

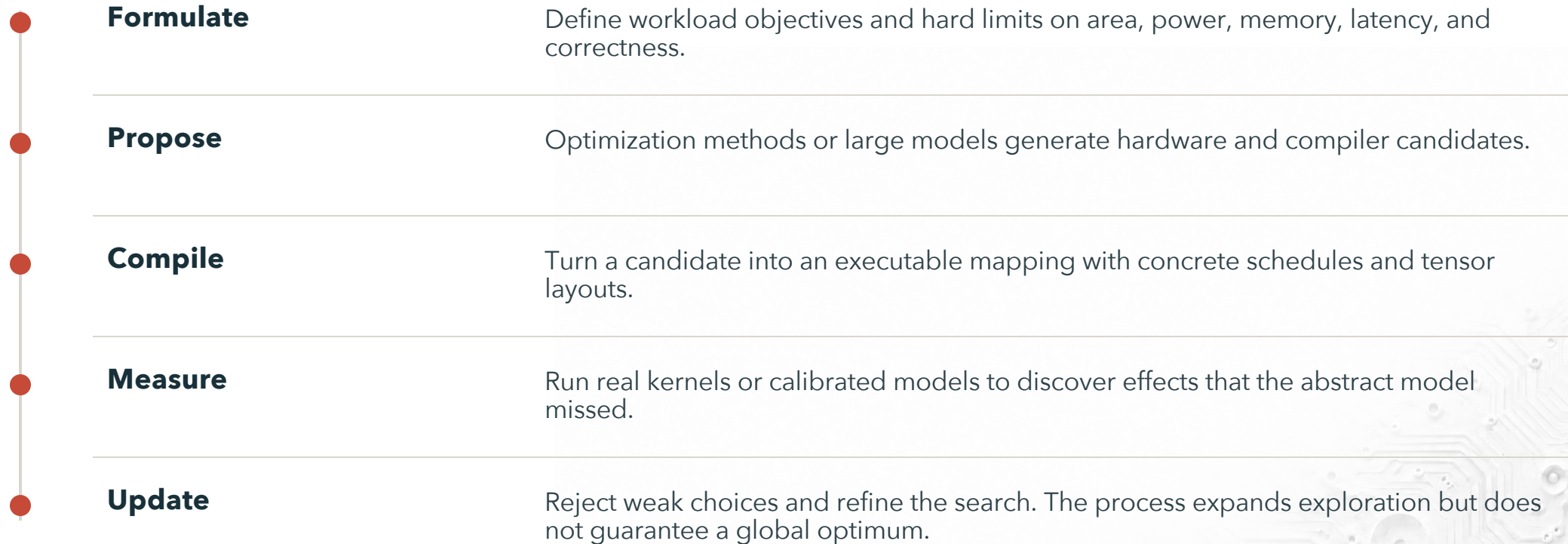
Hardware resources interact with tiling, fusion, tensor layout, scheduling, and model parallelism.

Useful throughput, task latency, energy, cost, and correctness determine whether a configuration works.

Co-design may combine analytical models, discrete optimization, simulation, real measurement, and expert constraints.

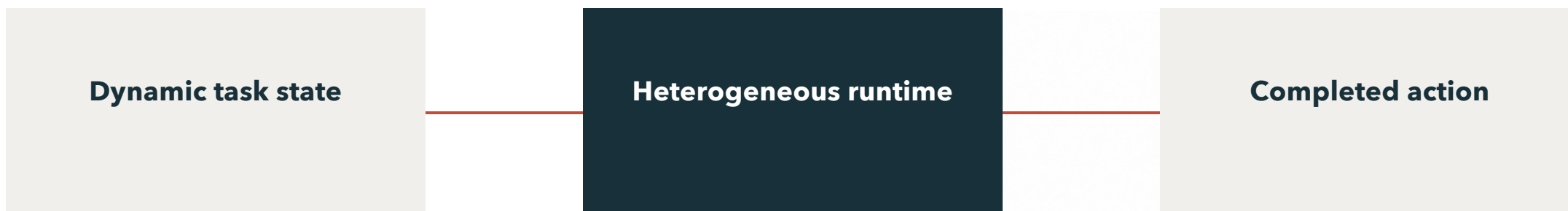
Automated search expands the co-design space

The reported Jalapeño effort illustrates a faster evidence loop across hardware and software.



Agentic AI changes the workload again

The execution path now depends on intermediate results and external systems.



A request can trigger planning, retrieval, code execution, database access, or a remote service call.

CPU control, accelerator inference, storage, networks, and tools alternate rather than forming one regular tensor graph.

The system must preserve state and schedule the next step without leaving expensive hardware idle while tools respond.

Some agents remain model-dominated. Others spend substantial time in retrieval, tools, communication, or waiting.

Useful performance is harder to define

Peak FLOPS cannot describe whether an agent completes a task well.

Task-level latency

Measure time to the first useful response and time to complete the full task, including tools and retries.

Energy and cost

Report energy or cost per successful task rather than only per generated token.

Correctness

A faster wrong answer has little value. Performance metrics must retain the required quality and verification level.

Adaptive systems must remain predictable

Dynamic scheduling can improve utilization, but it also creates new failure modes.

Adaptation

Schedulers may change batching, model choice, numerical precision, cache placement, or CPU and accelerator assignment.

Service guarantees

Adaptive decisions must respect latency, isolation, correctness, and safe operating limits.

Observability

Engineers need to see which resource caused a delay and why the system changed its decision.

Energy, memory, and communication will constrain scale

Moving data through a large system can cost more than the arithmetic itself.

Data movement

Weights, attention state, gradients, and messages travel through caches, high-bandwidth memory, and networks.

Physical limits

Cooling, power delivery, memory supply, and data-center construction can become binding constraints.

System response

Use suitable precision, keep data near its computation, and match network topology to communication patterns.

THE QUESTION TO KEEP ASKING

Which resource does the workload leave waiting?

To anticipate the next architecture, identify the repeated work, the data that moves most often, the dependency that prevents parallel execution, and the resource that remains idle. Those answers reveal the current bottleneck and the hardware-software change most likely to matter.